Pricing the American put via PSOR

Roberto Cepeda Rocandio

May 2026; revised October 2026

1 Setting and motivation

The American put can be exercised at any time $t \in [0, T]$, not only at maturity. This single change relative to the European case transforms the pricing problem qualitatively: the price now depends on a free *exercise boundary* $S_f(t)$ that separates a *continuation region* (where the option is held) from an *exercise region* (where it is exercised immediately). The boundary itself is unknown a priori — it is part of the solution.

The deliverable of this note is a numerical method that handles the free boundary cleanly, by reformulating the problem as a *Linear Complementarity Problem* (LCP) on a fixed domain and solving it iteratively at every time step using *Projected Successive Over-Relaxation* (PSOR). The implementation consists of two modules:

- `quantlib.psor`: a generic PSOR solver for tridiagonal LCPs (analogous to `quantlib.thomas`).
- `quantlib.cn_american`: the user-facing American put pricer, built on the θ-scheme infrastructure (`quantlib.theta_scheme`) with PSOR replacing Thomas in the per-step solve.

2 The American put as an optimal stopping problem

Under the risk-neutral measure $\mathbb{Q}$, the price of the American put is

$$V(S_0, 0) = \sup_{\tau \in \mathcal{T}_{[0,T]}} \mathbb{E}^{\mathbb{Q}}\big[\, e^{-r\tau}(K - S_\tau)^+ \,\big|\, S_0 \,\big], \tag{1}$$

where $\mathcal{T}_{[0,T]}$ is the set of stopping times taking values in $[0, T]$. Equation (1) states that the holder picks the stopping rule τ that maximises the discounted expected payoff.

The optimal τ partitions the state space (S, t) into two regions:

- **Continuation region** $\mathcal{C} = \{(S, t) : V(S, t) > (K - S)^+\}$: holding the option is strictly better than exercising now.
- **Exercise region** $\mathcal{E} = \{(S, t) : V(S, t) = (K - S)^+\}$: it is optimal (or at least no worse) to exercise.

The boundary between the two is the curve $S = S_f(t)$, the *exercise boundary*, also called the *free boundary* because its location is part of the solution rather than known a priori. For a put, $S_f(t)$ is increasing in t with $S_f(T^-) = K$.

3 The classical free-boundary PDE formulation

In the continuation region, the standard arbitrage argument shows that V satisfies the Black–Scholes PDE:

$$\frac{\partial V}{\partial t} + \tfrac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} + rS\frac{\partial V}{\partial S} - rV = 0, \qquad (S, t) \in \mathcal{C}. \tag{2}$$

In the exercise region, V equals the intrinsic value:

$$V(S,t) = K - S, \qquad (S,t) \in \mathcal{E}. \tag{3}$$

At the free boundary $S = S_f(t)$, two matching conditions hold:

$$V(S_f(t),t) = K - S_f(t), \tag{4}$$

$$\frac{\partial V}{\partial S}(S_f(t),t) = -1. \tag{5}$$

The first is value matching: V is continuous across the boundary. The second is the *smooth pasting* condition: V_S is also continuous. Two conditions are needed because the boundary itself is unknown; smooth pasting is the optimality condition that determines $S_f(t)$.

Why this formulation is hard numerically. The PDE (2) holds on the time-varying domain $\mathcal{C}$, whose boundary moves with t. The finite-difference methods for European options (FTCS, BTCS, CN) work on fixed domains; adapting the spatial mesh to track $S_f(t)$ would require remeshing at every time step, which is expensive and error-prone. We need a reformulation on a fixed domain.

4 The variational reformulation as an LCP

Switching to log-coordinates $x = \ln(S/K)$ and $\tau = T - t$ and denoting the spatial operator $\mathcal{L}V = \frac{\sigma^2}{2}V_{xx} + \mu V_x - rV$ as in the Crank–Nicolson note, we reformulate the American put as the system

$$\boxed{\begin{aligned} V_\tau - \mathcal{L}V &\geq 0 \\ V - g &\geq 0 \\ (V_\tau - \mathcal{L}V)(V-g) &= 0 \end{aligned}} \qquad \text{on } x \in (x_{\min}, x_{\max}),\ \tau \in (0,T], \tag{6}$$

where $g(x) = K(1-e^x)^+$ is the put payoff function (the *obstacle*). Initial condition: $V(x,0) = g(x)$. Boundary conditions: $V(x_{\min},\tau) = K - Ke^{x_{\min}}$ (deep ITM: exercise immediately, no discount) and $V(x_{\max},\tau) = 0$ (deep OTM: worthless).

Why each inequality is correct. Take them in turn.

1. $V_\tau - \mathcal{L}V \geq 0$. In $\mathcal{C}$ this is an equality (the BS PDE itself). In $\mathcal{E}$ we have $V \equiv g$, so $V_\tau = 0$, and a direct calculation $\mathcal{L}g = -rK \leq 0$ for $x < 0$ (the put-ITM region) makes $V_\tau - \mathcal{L}V = -\mathcal{L}g \geq 0$.

2. $V - g \geq 0$. The price never falls below the immediate payoff: if it did, exercising would dominate holding, an arbitrage.

3. $(V_\tau - \mathcal{L}V)(V-g) = 0$ *(complementarity)*. At every (x,τ), exactly one of the two preceding inequalities is strict: in $\mathcal{C}$ the second is strict ($V > g$) and the first is an equality (BS PDE); in $\mathcal{E}$ the first is strict ($V_\tau - \mathcal{L}V > 0$) and the second is an equality ($V = g$).

The structural payoff. The system (6) makes no reference to the free boundary S_f. The boundary emerges a posteriori as the locus where one inequality changes from strict to equality. Numerically, this is enormous: we can work on the fixed log-truncated domain of the finite-difference foundations note, with no need to track a moving boundary, and the complementarity condition automatically selects which nodes are in which region.

Equivalence. The LCP (6) is equivalent to the classical PDE+free-boundary formulation (2)–(5) for solutions $V \in C^1$. The smooth pasting condition is recovered automatically: from $V \geq g$ everywhere and $V = g$ on $\mathcal{E}$, taking the spatial derivative at the boundary gives $V_S \geq g_S = -1$ from the continuation side and $V_S = -1$ on the exercise side; combined with $V \in C^1$, this forces $V_S = -1$ at the boundary. Reference: Wilmott–Howison–Dewynne [1], Chapter 7.

5 Discretisation: from continuous to discrete LCP

Apply the θ-scheme to the operator $V_\tau - \mathcal{L}V$ exactly as in the Crank–Nicolson note. The discrete equality $AV^{n+1} = BV^n + \mathbf{b}_{\text{bnd}}$ (with A, B, $\mathbf{b}_{\text{bnd}}$ from that note) becomes the discrete LCP

$$\boxed{\begin{aligned} AV^{n+1} &\geq BV^n + \mathbf{b}_{\text{bnd}} \\ V^{n+1} &\geq \mathbf{g} \\ \left(AV^{n+1} - BV^n - \mathbf{b}_{\text{bnd}}\right) \cdot \left(V^{n+1} - \mathbf{g}\right) &= 0 \end{aligned}} \tag{7}$$

componentwise, where $\mathbf{g} \in \mathbb{R}^{N-1}$ is the payoff sampled on the interior grid: $g_j = K(1 - e^{x_j})^+$.

Each time step now requires solving the LCP (7) instead of a linear system. Thomas does not apply: it solves $Ax = b$, not $Ax \geq b$ with $x \geq c$. We need a new solver.

6 SOR: the iterative method behind PSOR

For the linear system $Ax = b$ with A tridiagonal and diagonal dominant, an alternative to Thomas is the iterative *Successive Over-Relaxation* (SOR) method. SOR produces a sequence $x^{(0)}, x^{(1)}, x^{(2)}, \ldots$ that converges to the exact solution. Although asymptotically more expensive than Thomas ($O(N\sqrt{\alpha}\,\log\epsilon^{-1})$ per solve at the optimal relaxation parameter, with α the mesh ratio defined below, against $O(N)$; see the section on computational cost), SOR has a structural advantage: it generalises naturally to LCPs.

Definition. SOR updates one component at a time. For a tridiagonal system $a_{i,i-1}x_{i-1} + a_{ii}x_i + a_{i,i+1}x_{i+1} = b_i$, SOR uses

$$x_i^{(k+1)} = (1-\omega)\,x_i^{(k)} \;+\; \omega \cdot \frac{1}{a_{ii}}\Big(\,b_i \;-\; a_{i,i-1}\,x_{i-1}^{(k+1)} \;-\; a_{i,i+1}\,x_{i+1}^{(k)}\,\Big) \tag{8}$$

for $i = 1, \ldots, n$, with $\omega \in (0, 2)$ the *relaxation parameter*. At $\omega = 1$, equation (8) is plain Gauss–Seidel; at $\omega > 1$ it over-relaxes (extrapolates beyond the GS update); at $\omega < 1$ it under-relaxes (interpolates between old and new). Note that (8) uses $x_{i-1}^{(k+1)}$ (the just-updated value) and $x_{i+1}^{(k)}$ (the old value) — this in-place update is what makes SOR efficient.

Convergence. A classical result (Kahan; Ostrowski–Reich):

Theorem 6.1 (SOR convergence)**.** *If A is symmetric positive definite, SOR converges for every $\omega \in (0, 2)$ from any initial guess $x^{(0)}$. For a tridiagonal A with constant coefficients (our case), the convergence rate is geometric with optimal relaxation parameter*

$$\omega^* = \frac{2}{1 + \sqrt{1 - \rho_J^2}}, \tag{9}$$

where ρ_J is the spectral radius of the Jacobi iteration matrix $D^{-1}(L + U)$.

For the matrices arising from BTCS or CN, ρ_J is governed by the mesh ratio $\alpha = \sigma^2\Delta\tau/(2\Delta x^2)$: ignoring the small convective asymmetry, $\rho_J \approx 2\theta\alpha/(1+2\theta\alpha) = 1 - O(1/\alpha)$. Refining the grid with $M \propto N$ makes $\alpha \propto N$, so $\omega^* \to 2$ like $2 - O(1/\sqrt{N})$. For the grids used here it stays well below 2: for Crank–Nicolson ($\theta = 1/2$), $\rho_J \approx 0.76$ and $\omega^* \approx 1.21$ at $N = 200$, $M = 100$, and $\rho_J \approx 0.86$, $\omega^* \approx 1.33$ on the 400×200 benchmark grid.

Asymmetry. The CN matrix A is not symmetric (the convective term μV_x uses centred differences, breaking symmetry). The Ostrowski–Reich theorem does not apply directly. A is a strictly diagonally dominant M-matrix, but for such matrices (and H-matrices in general) the general guarantee is only $0 < \omega < 2/(1+\rho(|J|))$, with J the Jacobi iteration matrix (see Varga [8]). The full range $\omega \in (0,2)$ holds here for a different reason. With the CN coefficients of the Crank–Nicolson note, the two off-diagonals have the same sign, $\beta_-\beta_+ = (\alpha^2 - \nu^2)/4 > 0$ whenever the grid Péclet number $|\nu|/\alpha = |\mu|\Delta x/\sigma^2$ is below one, so a diagonal scaling D with $d_{i+1}/d_i = \sqrt{\beta_-/\beta_+}$ makes $D^{-1}AD$ symmetric, with off-diagonals $-\sqrt{\beta_-\beta_+}$ and the same diagonal; being strictly diagonally dominant, it is positive definite. The scaling preserves the splitting into diagonal, lower and upper parts, so the SOR iteration matrices of A and $D^{-1}AD$ are similar, and Ostrowski–Reich gives convergence for every $\omega \in (0,2)$; Young's formula (9) then holds with ρ_J computed from A. In practice the projected iteration has its optimum close to the linear prediction on the small test problem and above it on the benchmark grid: $\omega = 1.2$ on the 100-unknown test problem of `validate_psor.py` (linear theory: 1.25), and about 1.5–1.6 on the 400×200 benchmark grid (linear theory: 1.33; sweep counts printed by test [4] of `validate_cn_american.py`). The default $\omega = 1.2$ is a safe choice rather than the optimum; `benchmark_phase3.py` uses 1.4.

7 PSOR: SOR with projection

PSOR is the generalisation of SOR to linear complementarity problems. The modification is a single line: after each SOR component update, project onto the constraint $x_i \geq g_i$:

$$\boxed{x_i^{(k+1)} = \max\left\{ g_i,\ \ (1-\omega)\, x_i^{(k)} + \omega \cdot \frac{1}{a_{ii}}\Big(b_i - a_{i,i-1}\, x_{i-1}^{(k+1)} - a_{i,i+1}\, x_{i+1}^{(k)}\Big) \right\}.} \tag{10}$$

That is: compute the SOR candidate, and if it falls below the obstacle g_i, raise it to g_i. If it lies above, leave it.

Variational interpretation. The LCP (7) is equivalent (when A is symmetric positive definite) to the constrained minimisation

$$\min_{x \geq \mathbf{g}} \ \ \tfrac{1}{2} x^T A x - x^T b,$$

where $b = BV^n + \mathbf{b}_{\text{bnd}}$. PSOR is the componentwise application of SOR to the KKT conditions of this minimisation, and the projection $\max\{g_i, \cdot\}$ is exactly the projection onto the feasible set $\{x : x \geq \mathbf{g}\}$. This construction is due to Cryer [5].

Theorem 7.1 (Cryer 1971)**.** *If $A \in \mathbb{R}^{n\times n}$ is symmetric positive definite and $\omega \in (0,2)$, the PSOR iteration* (10) *converges to the unique solution of the LCP $Ax \geq b$, $x \geq \mathbf{g}$, $(Ax - b)(x - \mathbf{g}) = 0$ from any initial guess $x^{(0)} \geq \mathbf{g}$.*

For non-symmetric A. The CN matrix is not symmetric, but is a strictly diagonally dominant M-matrix in the project's parameter range. PSOR convergence in this setting follows from results of Mangasarian and others; see Cottle–Pang–Stone [6], Chapter 5, for the rigorous treatment. The qualitative picture is that for moderate Péclet number, asymmetry slows convergence by a constant factor but does not destroy it.

Initialisation. A natural choice is $x^{(0)} = \mathbf{g}$ (the obstacle itself). It is feasible by construction, and is exact on the eventual exercise region. An alternative is warm-starting from the previous time step, $x^{(0)} = V^n$. Since $V^{n+1} - V^n = O(\Delta\tau)$, the warm start begins much closer to the solution and needs fewer sweeps; it is what the implementation uses.

8 Stopping criterion

The iteration must be terminated at finite k. The standard test is on the increment:

$$\left\|x^{(k+1)} - x^{(k)}\right\|_\infty < \texttt{tol}_{\text{abs}} + \texttt{tol}_{\text{rel}} \cdot \left\|x^{(k)}\right\|_\infty, \tag{11}$$

with default values $\texttt{tol}_{\text{abs}} = 10^{-10}$ and $\texttt{tol}_{\text{rel}} = 10^{-9}$. The absolute component protects against deep-OTM puts where prices are small (cents); the relative component protects against deep-ITM puts where prices are large (close to K). The mixed criterion is robust across the entire strike range.

Why not the residual. The residual criterion $\|Ax - b\| < \texttt{tol}$, standard for linear systems, does not apply to the LCP: the residual is non-zero in the exercise region (by the first inequality) even at the optimum. The increment criterion is agnostic to the problem type and works for both linear systems and LCPs.

Maximum iteration safeguard. If ω is mis-tuned or the matrix is poorly conditioned, the iteration may converge slowly or not at all. A hard limit `max_iter` (typically 10^4) prevents an infinite loop. Reaching the limit raises a `RuntimeError` (or `std::runtime_error` in C++), making a silent failure visible to the caller.

9 Computational cost

Per time step:

- Each PSOR sweep is $O(N)$ operations.
- Number of sweeps to reach tolerance ϵ: $K \sim -\log\epsilon / \log(1/\rho(\omega))$. At the optimum $\rho(\omega^*) = \omega^* - 1 = 1 - O(1/\sqrt{\alpha})$, so $K = O(\sqrt{\alpha}\,\log(1/\epsilon))$.
- Cost per time step: $O(NK)$.

With $M \sim N$ (matched to spatial accuracy as in CN), $\alpha \propto N$ and the total cost is $O(N^{5/2})$ up to the logarithm (at $\omega = 1.4$ the mean number of sweeps per step grows from 13.8 on the 200×100 grid to 20.3 on 400×200, a ratio of $1.47 \approx \sqrt{2}$; test [4] of `validate_cn_american.py`). Measured on the 400×200 benchmark grid with the default tolerances, PSOR needs about 30 sweeps per time step at $\omega = 1.2$ and 18 at $\omega = 1.5$, against one Thomas solve for the European pricer: a few tens of times more work per step, the price of LCP generality.

Comparison with other American pricers.

Method	Cost	Accuracy
CRR binomial (`quantlib.american`)	$O(N^2)$	$O(1/N)$, with oscillations
LSM (`quantlib.american`)	$O(N_{\text{paths}} N_{\text{steps}})$	MC, $O(1/\sqrt{N_{\text{paths}}})$
CN-PSOR (this note)	$O(N^{5/2} \log\epsilon^{-1})$, $M \propto N$	$O(\Delta x^2) + O(\Delta\tau)$

CN-PSOR is the most accurate but most expensive at moderate N. LSM scales best to high dimensions; CRR is the simplest and most robust. The finite-difference/lattice benchmark (`benchmark_phase3.py`) makes the comparison numerical.

10 Implementation outline

Two modules:

`quantlib.psor` / `quant::pde::psor`. Generic LCP solver. The signature mirrors Thomas:

```
def psor_solve(sub, diag, sup, rhs, obstacle, *,
               omega=1.2, tol_abs=1e-10, tol_rel=1e-9,
               max_iter=10000, x0=None) -> (x, n_iter)
```

Returns both the solution and the number of iterations performed (the iteration count is diagnostic data; `validate_psor.py` uses it in its ω sweep).

`quantlib.cn_american` / `quant::pde::cn_american`. American put pricer. Reuses the θ-scheme grid and boundary construction of the Crank–Nicolson pricer, replacing the per-step Thomas solve with a PSOR solve. Two notable differences from `cn_european_put`:

1. The lower boundary condition is $V(x_{\min}, \tau) = K - Ke^{x_{\min}}$ (no discount, since exercise is immediate).

2. No Rannacher smoothing. The Rannacher protocol addresses the kink in the European payoff, which produces high-frequency oscillations that CN cannot damp. For the American put the oscillations are the same in nature, and the projection step clips them from below: nodes near the strike fall in the exercise region $\mathcal{E}$ where $V = g$ exactly. This does not remove them entirely: at $N = 400$ and coarse M the price still oscillates (5.9407 at $M = 12$, 6.1373 at $M = 25$, 6.0840 at $M = 50$, against ≈ 6.0899). Nor does CN keep its second order in time. At fixed $N = 400$, with PSOR tolerance 10^{-14} (test [8] of `validate_cn_american.py`), the successive differences of the price for $M = 100 \to 200 \to 400 \to 800$ are 1.70×10^{-4}, 7.02×10^{-5} and 3.37×10^{-5} (ratios 2.42 and 2.08): first order in $\Delta\tau$. The early-exercise constraint puts a kink in time into the solution (where the free boundary crosses a node, V is not smooth in τ), and that limits CN to $O(\Delta\tau)$ in practice.

11 Validation strategy

Eight tests certify the new pricer. All but the ω-sweep are in `validate_cn_american.py`; the sweep is in `validate_psor.py`, and `test_cn_american.cpp` repeats the CRR, dominance, refinement and input checks for the C++ pricer.

1. **Cross-validation against CRR binomial**: the gold-standard reference for American puts, here a self-contained 2000-step CRR tree embedded in the test files (independent of `quantlib.american`). Seven parameter sets at $N = 200$, $M = 100$, tolerance 5×10^{-3}.

2. **Refinement reduces the error**: against a 5000-step CRR price, the error must decrease from $(N, M) = (100, 50)$ to $(200, 100)$, and be below 5×10^{-3} at $(400, 200)$. Only the decrease is checked, not the rate (the validator's docstring expects $O(h)$ near the free boundary); the printed error ratios are 3.80 and 4.37.

3. **American $\geq$ European**: at every spot, the American put price must be at least the European put price. Strict in the exercise region, equality possible deep in the continuation region.

4. **ω-sweep convergence**: at fixed problem, $\omega \in \{0.5, 0.8, 1.0, 1.2, 1.4, 1.6, 1.8\}$ produces different iteration counts with an interior minimum. It occurs at $\omega = 1.2$, in line with the linear-theory value $\omega^* \approx 1.25$ for that test matrix.

5. ω **outside** $(0, 2)$ **raises**: `psor_solve` checks $0 < \omega < 2$ before iterating and raises a `ValueError` for $\omega \in \{0, 2, -0.5, 2.5\}$.

6. **Free boundary**: the recovered exercise boundary S_f (the largest S in the money with $V = g$) lies below K, and is closer to K for the shorter maturity: $S_f = 86.94$ at $T = 0.25$ against 80.57 at $T = 1$. With τ the time to maturity, $S_f(\tau)$ decreases in τ and tends to K as $\tau \to 0$; the full curve is in `free_boundary.csv` of `benchmark_phase3.py` (98.78 at $\tau = 0.00125$ on the trinomial trace, decreasing thereafter). The validator compares the two maturities only: a qualitative shape test that catches gross errors.

7. **Input validation**: $S = 0$ and a spot outside the truncated domain raise.

8. **Order in time**: at fixed $N = 400$, the successive price differences for $M = 100, 200, 400, 800$ must shrink by a factor between 1.5 and 3 (first order in $\Delta\tau$); the measured ratios are 2.42 and 2.08. The prices at $M = 12, 25, 50$ are printed to show the residual oscillation.

References

References

[1] P. Wilmott, S. Howison and J. Dewynne. *The Mathematics of Financial Derivatives.* CUP, 1995. Chapter 7 on American options as free-boundary problems; self-contained derivation of smooth pasting from optimality.

[2] P. Wilmott. *Paul Wilmott on Quantitative Finance.* 2nd ed., Wiley, 2006. Chapters 9 and 63 on American options; Chapters 77–78 (finite-difference methods for one-factor models) and 82 (finite-difference programs).

[3] D. Tavella and C. Randall. *Pricing Financial Instruments: The Finite Difference Method.* Wiley, 2000. Pricing equations and the relationship between European and American derivatives; analysis of finite-difference schemes (Chapter 2); Chapter 5 (Numerical Examples) works through applications.

[4] R. Seydel. *Tools for Computational Finance.* 6th ed., Springer, 2017. Chapter 4 on American options as obstacle problems and LCPs; concise.

[5] C. W. Cryer. *The solution of a quadratic programming problem using systematic overrelaxation.* SIAM Journal on Control 9, 385–392, 1971. Original PSOR paper: proof of convergence for symmetric positive-definite LCPs.

[6] R. W. Cottle, J.-S. Pang and R. E. Stone. *The Linear Complementarity Problem.* SIAM Classics in Applied Mathematics, 2009 (orig. Academic Press, 1992). The rigorous reference for LCP theory. Chapter 5 (Iterative Methods) covers iterative methods for the LCP, projected SOR among them.

[7] R. Glowinski. *Numerical Methods for Nonlinear Variational Problems.* Springer, 2008 (orig. Springer 1984). The variational treatment of obstacle problems.

[8] R. S. Varga. *Matrix Iterative Analysis.* Revised 2nd ed., Springer, 2000. Chapters 3–4 on SOR convergence theory and optimal ω.

[9] Y. Saad. *Iterative Methods for Sparse Linear Systems.* 2nd ed., SIAM, 2003. Chapter 4 on relaxation methods, modern treatment.